

17-214/514: Agentic Software Development

APIs, Libraries, and Frameworks

Hammad Ahmad and Claire Le Goues • Monday, October 5, 2026 • **Lecture**

What did we talk about last time?

- A pattern has a name, a problem, a solution, and consequences. We have seen Strategy, Observer, and Factory.
- Open/Closed says to extend through designed extension points instead of editing working code.
- A pattern usually costs some indirection. Argue the tradeoff in terms of quality attributes.
- A pattern name is a checkable instruction to an agent. When an agent adds a pattern, check that it solves a problem you have.
- Today: let's switch gears. What happens when other people's code depends on yours.

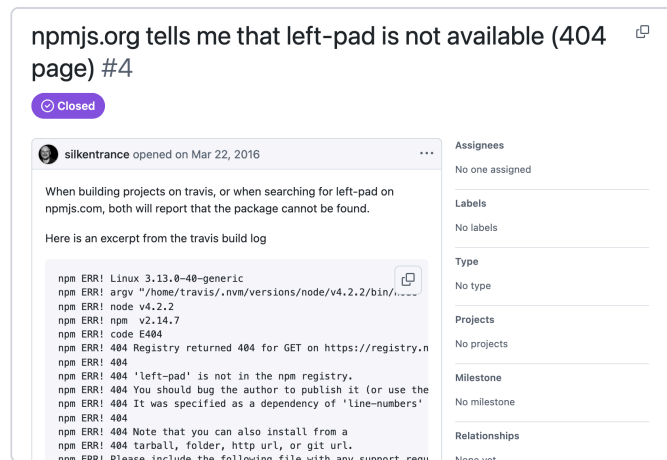
Learning goals for today

- Design an API that is easy to use correctly and hard to misuse, for human and agent callers
- Use types as machine-checked specifications that make illegal states unrepresentable
- Evaluate whether an implementation honors its contract (substitutability), and verify a boundary with a contract test
- Evolve an interface without breaking its consumers
- Tell a library from a framework, and decide when to reuse one and when to let an agent generate the code

Why is a stranger angry with you?

You renamed one parameter and swapped two others. Your tests pass. Someone you have never met now has a broken build.

- Inside your codebase, a signature change is a refactor. The IDE can update every caller.
- Callers outside your codebase are different. You cannot see them or update them, and you may not know they exist.
- We will focus on that difference today.



github.com/left-pad/left-pad/issues/4

Contracts with strangers

An API is a promise made to strangers

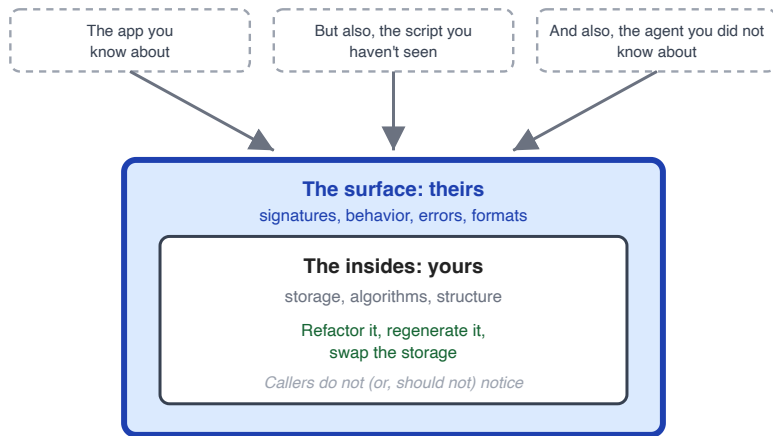
- An API is a contract. You promise behavior, and callers build on the promise.
- Treat callers as strangers (unknown people with unknown uses). Fixing their code when you change yours is hard.
- You have been writing contracts for weeks. In A2, you built an API for Better Slack.
 - Its callers now include a client you do not control. A change on your side can break something on theirs.

As an aside: REST APIs

- A REST-style API is a set of URLs that clients call over HTTP. You have repaired one (Bad Slack) and designed one (Better Slack).
- Each call names a resource with a URL and an action with a method. GET reads, POST creates, PUT replaces, DELETE removes.
- Data travels as JSON in the request and response bodies. The result is a status code (200 fine, 201 created, 404 not found).
- The server keeps no memory of the conversation. Every request carries what it needs, the token included.

The boundary between theirs and yours

- The surface (signatures, behavior, errors, formats) belongs to your callers. Change it with *a lot of care*.
- Everything behind the surface belongs to you. Refactor it, regenerate it, swap the storage, etc.
- Usually, the boundary decides what you can change without asking anyone.
- So mark what is public (exports, naming, `@internal`, docs). An interface that is public by accident can collect callers by accident.



A change to the surface needs agreement from callers you cannot see

Why does `make` treat tabs and spaces differently?

Why the tab in column 1? Yacc was new, Lex was brand new. I hadn't tried either, so I figured this would be a good excuse to learn. After getting myself snarled up with my first stab at Lex, I just did something simple with the pattern newline-tab. It worked, it stayed. And then a few weeks later I had a user population of about a dozen, most of them friends, and I didn't want to screw up my embedded base. The rest, sadly, is history.

- Stuart Feldman, who wrote `make` in 1976, quoted in Eric S. Raymond, *The Art of Unix Programming* (2003)

Who calls your API in 2026?

Humans

read your docs and copy your examples

Tools

generate whole clients from a machine-readable description of your API

Agents

write calling code from your names, types, and examples. That is their context.

An API that is hard for a human to misuse is generally hard for an agent to misuse.

API usability

An API should be easy to use correctly

- Names that say what things do, and conventions that stay consistent.
 - When the conventions drift, callers may see multiple names for one idea (e.g., `getUser`, `fetchRoom`, and `loadBooking` for the same kind of lookup).
- Sensible defaults, so the simple case is simple.
 - e.g., `fetch(url)` is a GET with sane headers. You only pass `{ method: "POST" }` when you want something other than a GET.
- The obvious way to call it should be the correct way.
 - e.g., Java's try-with-resources made closing a file the obvious shape, where the old `finally { close(); }` was easy to get wrong.

Callers guess from consistency

- If your API follows a pattern, callers assume the pattern holds. Two verbs and two nouns means they will try all four combinations.
 - e.g., Java 1.0 had a wrapper class for six of its eight primitive types (`Integer` , `Long` , `Float` , `Double` , `Character` , `Boolean`), so callers assumed `Byte` and `Short` existed too. Both arrived in 1.1.
- Do not make callers choose along two axes at once.
 - e.g., Java's `Runnable` (for threading) returns nothing and throws nothing. `Callable` returns a value and may throw. "Returns nothing, may throw" has no standard interface.
- An agent completes a pattern kind of like a person does, just faster.
 - The compiler catches a hallucinated name, but not a guess that happens to be a real method with a different meaning.

Docs are part of the API

- Your examples may get copied verbatim, since your examples are their context.
- A wrong example in the docs is a bug that gets copied.
 - e.g., the forum snippet that makes an Android app accept every TLS certificate spread far enough that Google Play began rejecting apps that contained it (2016).
- Use helpful error messages. "expected ISO-8601 date, got epoch millis" helps. "invalid input" does not.
 - e.g., Rust's compiler errors (what went wrong, where, and a fix to try) are the model many tools now copy.
- Test the docs the way you test code. Hand them to an agent with no other context and read the client code it writes.
 - e.g., back in the day, Bloch's advice was to write to your API before it exists. An agent is a client that can do that very quickly.

An API should be hard to misuse

- Hard to use incorrectly is a design property (Bloch's bar for an API).
 - e.g., `Thread.sleep(1000)` takes any number, and you need to remember (or look up) that it is milliseconds. `Thread.sleep(Duration.ofSeconds(1))` (Java 19) cannot be handed the wrong unit.
- The strongest form: the wrong call cannot be written at all.
 - e.g., before Java 5 a color was an `int` constant, so `setColor(42)` compiled. With an enum, only the colors that exist can be passed.

Famous examples from Java

```
Calendar c = Calendar.getInstance();
c.set(2026, 10, 5);           // November 5. Months start at 0.
Date d = new Date(126, 9, 5); // October 5, 2026. Years start at 1900.

List<Integer> ids = new ArrayList<>(List.of(7, 3, 1));
ids.remove(1);           // removes the element at index 1
ids.remove(Integer.valueOf(1)); // removes the element equal to 1

if (Thread.interrupted()) { ... } // a question that also clears the flag
```

- Every line compiles, but several do something other than what they look like.
- If you are curious, `java.time` (2014) fixed the dates with `LocalDate.of(2026, Month.OCTOBER, 5)`. The wrong month is no longer writable.

An API that is easy to misuse

```
function book(a: string, b: string, c: number, d: number,  
             flag?: boolean): any
```

```
book(userId, roomId, end, start, true);  
// compiles, runs, books something?
```

- Two interchangeable strings and two interchangeable numbers, so every argument-order mistake compiles.
- Milliseconds or seconds? The type does not say.
- What is `flag`? And `any` promises nothing about the result.

The redesign for `book()`

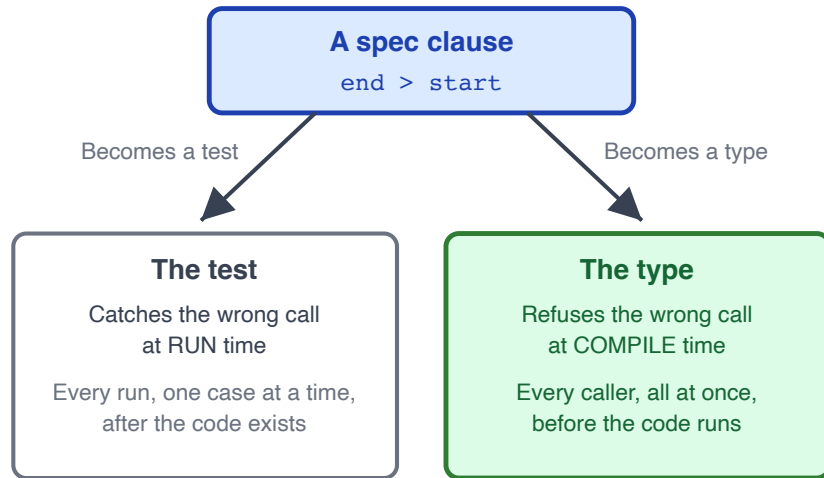
```
function book(request: BookingRequest): Booking

interface BookingRequest {
  user: UserId;
  room: RoomId;
  slot: TimeRange;    // start/end validated at construction
}
```

- `UserId` and `RoomId` cannot be swapped. The compiler will scream at you.
- The interval cannot be backwards, because `TimeRange` will not construct one.
- No flag and no `any`. The return type says what you get.
- The functionality is the same, and the misuses no longer compile.

Types as machine-checked specs

- A spec's clauses became tests. Some clauses can become types instead, and then a violation does not compile.
 - Over a network there is no compiler. Validation at the boundary is the nearest thing, at request time but before any logic.
- Try to make illegal states unrepresentable. If an invalid combination cannot be constructed, you no longer have to test for it.



The violation does not compile

*Same clause, two enforcement points.
Take the earlier one when the clause allows it*

The interval that cannot be backwards

```
class TimeRange {  
    private constructor(readonly start: Date, readonly end: Date) {}  
  
    static of(start: Date, end: Date): TimeRange {  
        if (end <= start) throw new InvalidRange(start, end);  
        return new TimeRange(start, end);  
    }  
}
```

- A private constructor and one validated way in, so a `TimeRange` that exists is valid.
- Every function that accepts a `TimeRange` inherits the guarantee, so it does not need to re-check the interval.
- The invariant lives (once) with the data it constrains. `TimeRange` holds both dates, so it can check them (Information Expert).

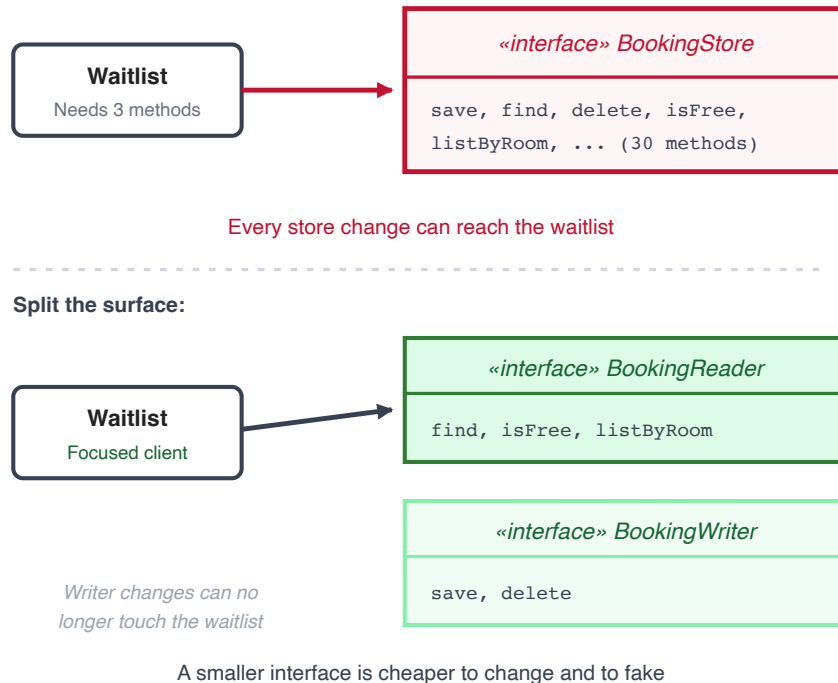
Errors are part of the contract

- What can go wrong, and how does the caller find out? That is as much of the contract as the happy path.
 - e.g., Better Slack's core contract fixed both. Every failure body is `{"error": ...}`, and a request that fails in several ways gets one code, checked in a fixed order (401, 404, 400, 409).
- Callers write `catch` blocks and `if (result.ok)` branches against your answers. Change the answers and the callers can break.

Back to SOLID

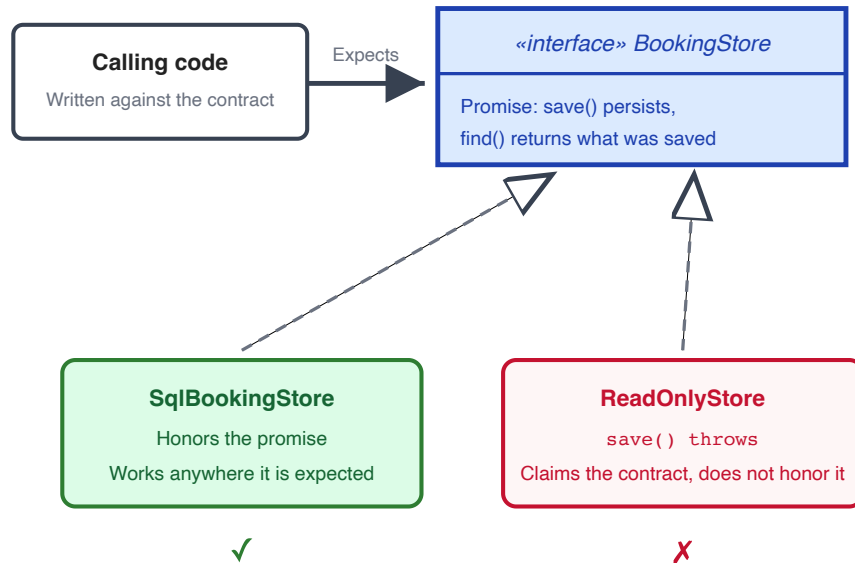
"I" = Interface segregation

- No client should have to depend on methods it does not use.
- If the waitlist needs three methods of a 30-method `BookingStore`, every change to the store can reach the waitlist for no reason.
- Split the surface, for example into `BookingReader` and `BookingWriter`. Smaller interfaces are also cheaper to fake.



"L" = Liskov substitution principle (LSP)

- Anything claiming a contract must honor it. A subtype must work anywhere its supertype is expected.
- Not "compiles when substituted" but "behaves when substituted". No stricter demands on callers, and no weaker promises back.
- Recall refused bequest, where a subclass inherits a method and overrides it to throw or do nothing. That is an LSP violation.



Both compile, one behaves

Tests guard this, types alone do not

A subtype that breaks the contract

```
class ReadOnlyStore implements BookingStore {  
    find(id: BookingId): Booking { /* fine */ }  
    save(b: Booking): void {  
        throw new NotSupportedError(); // the contract did not say this  
    }  
}
```

- It claims `BookingStore`, so every caller written against the contract can now fail at run time.
- It type-checks and still breaks the contract. LSP is about behavior, which is why you need tests to guard it.
 - e.g., a test for `BookingStore` (`save(b)`, then `find(b.id)` returns `b`). Run it against every class that implements `BookingStore`. `ReadOnlyStore` will fail on the first line.

That completes SOLID, if you're keeping track

S**Single
Responsibility**

one reason to
change (Week 2)

O**Open/Closed**
extend without
editing (last time)**L****Liskov
Substitution**
honor the contract
you claim (today)**I****Interface
Segregation**
small surfaces
(today)**D****Dependency
Inversion**
depend on
abstractions
(Week 2)

Evolving without breaking

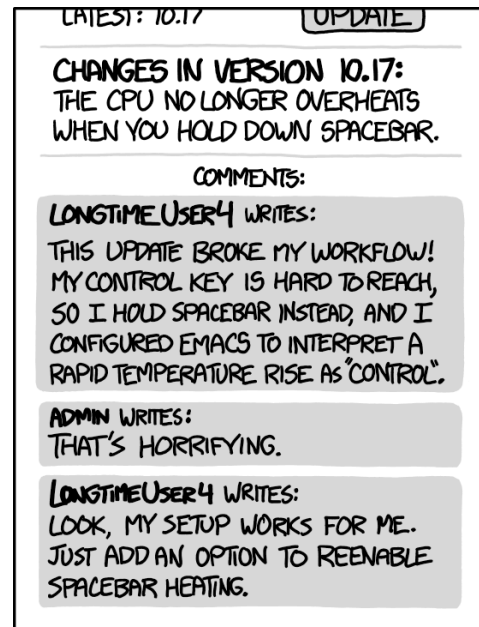
Hyrum's Law

With a sufficient number of users of an API, it does not matter what you promise in the contract: all observable behaviors of your system will be depended on by somebody.

- Hyrum Wright, hyrumslaw.com

Not only the documented contract. The error message text, the list order, the timing, the field that wasn't documented.

- Often, the "real" contract is usually bigger than the one you wrote.



EVERY CHANGE BREAKS SOMEONE'S WORKFLOW.

xkcd.com/1172

Who depends on it?

- We saw that the data model is usually the hardest to change, then the API, then the code.
 - That ranking assumes you can edit every caller.
- Once strangers depend on a surface, that surface becomes the hardest thing to change, whether it is a data format, an API, or a function signature.
 - For example, the Linux kernel changes its internal interfaces in most releases. Its interface to user programs has stayed stable since the 1990s, so old binaries still run.
 - As another example, IPv6 was standardized in 1998. In 2026, about half of Google's users reach it over IPv6. A wire format everyone speaks cannot change easily.
- Before you change something, ask who would have to change with you, and whether you can reach them.

What you can do about Hyrum's Law

- When in doubt, leave it out. A feature that is not there cannot be depended on.
 - This is also a good reason to constrain your agent from over-engineering. An endpoint or a field not in the spec is a surface a stranger can start depending on.
- Say what you do not promise. "The format of this output may change" is part of the contract too.
- Randomize what you do not promise. Go shuffles map iteration order on purpose, so that no program can come to rely on it.
- Before a change, ask the callers you can reach. For the ones you cannot, assume they rely on everything they can observe.
- When you do change something, write it down for the callers you cannot reach. What changed, who is affected, and what to do about it.
 - e.g., Python's "What's New" page for each release has a porting section.

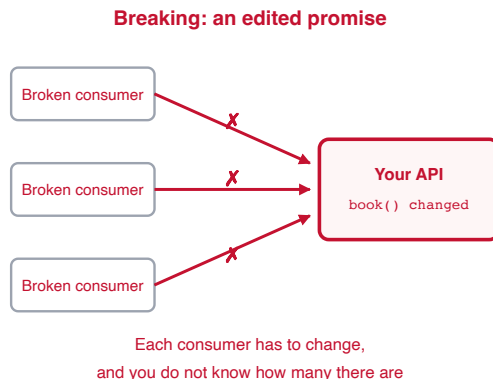
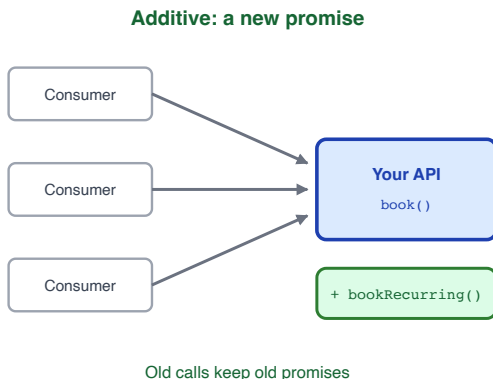
The contract will change and must not break

- Your API will almost certainly need to change, for new features, fixed mistakes, and moved requirements.
- The strangers built on the promise as it was.
- How do you change but not break?



Additive or breaking?

- **Additive** changes are safe. A new endpoint, a new optional field, a new error subtype that callers already catch generically.
- **Breaking** changes cost every consumer of that call. A rename, a removal, a retype, or a change in the meaning of an existing call.
- Semantic versioning (`major.minor.patch` for breaking, additive, fix) is a courtesy that often describes which kind you intended.



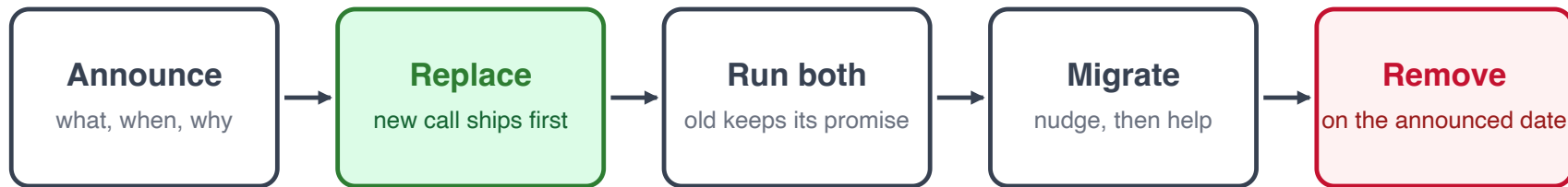
Recurring bookings arrive

Version 1 is live and has consumers: `book(request: BookingRequest): Booking`

- Tempting, but don't change `book()` to return `Booking[]`. Every existing caller breaks on the return type.
- Instead: add `bookRecurring(request, rule): RecurringBooking`, and let `book()` keep its meaning.
- Where possible, evolve by adding meanings rather than changing them. Old calls keep old promises.

Deprecation

Deprecation is a sequence of steps, each with a date.



- A `@Deprecated` mark with no removal date usually stays forever. Few people migrate away from a call that still works.
- A breaking change is easier to accept when the migration is mechanical.
 - For example, you can ship a script that rewrites the old call into the new one in the caller's code, or instructions an agent can follow to do the same.

Minor aside: the rules are the same over HTTP

Your HTTP handler calls a method, and everything so far applies to that method. At this boundary, there is no compiler, so:

In a method	At the HTTP edge of Better Slack
A type refuses the wrong call	Validation refuses a bad body, with a message that says why
Exceptions and result types	Status codes in a fixed order (401, 404, 400, 409)
A new parameter with a default	A new field or endpoint. Readers ignore unknown fields (the contract requires it of both sides)
A renamed method breaks the build	A renamed field breaks a client at run time, so it is a new version (the A3 question)
Callers depend on the error text	Clients depend on the error text, the field order, the undocumented field
The contract suite	Your harness, against your server and the staff servers

Let's practice: evolve without breaking

The live API and its consumer:

```
// your API, v1
cancel(id: BookingId): void

// a consumer you cannot edit
try { api.cancel(bookingId); showToast("Cancelled"); }
catch (e) { showError(e); }
```

New requirement: every cancellation now needs a **reason**. Some cancellations (e.g., late ones) also incur a fee, and the booking stays booked until the user has agreed to pay it.

Propose the evolved API. Then check whether that consumer still compiles **and still behaves correctly**.

Verifying the boundary

Contract tests

- A contract test runs against any implementation that claims the contract and does not care how the insides work.
 - Your A2 harness is one. It runs the same checks against your server and against our servers, so it is written against the contract.
- So it survives refactors and regenerations, which makes it the level to verify generated code at. (Check the boundary hard, the insides can change freely.)
- Contract tests are also LSP checks if you run the contract suite against a subtype.
- In A3 your API has a client you do not control, built from your own `CAPABILITY.md`. The production form is consumer-driven contracts, where downstream teams hand you their expectations as tests.

Which collaborators do you run for real?

- A contract suite runs against an implementation.
 - Which one? The real Postgres in a container, or an in-memory fake?
- Run the real thing where a fake would have to re-implement it. The database's constraints, the vendor's error codes, the file system's permissions.
- Fake what is slow, paid, not yours, or has effects outside the test (a real email to a real user, a charge to a credit card). Then run the contract suite against the fake too, so the fake does not deviate from the contract.
- A real database per test run is cheap now (e.g., using Testcontainers or docker compose). It starts empty, the suite seeds the data, and it is gone when the suite ends.

The API design review

Walk a proposed API (e.g., v2 for Better Slack) before it ships. In order:

- **Names:** Can a stranger guess what each call does? Would an agent?
- **Types:** What misuse is still expressible? Can a type remove it?
- **Errors:** What can go wrong, and how does the caller find out?
- **Evolution headroom:** Could a plausible next feature be added without changing an existing call?
- **Client code:** Have a fresh-context agent write calling code before the API ships, from the docs alone. What it gets wrong is the review.

Now you are the stranger

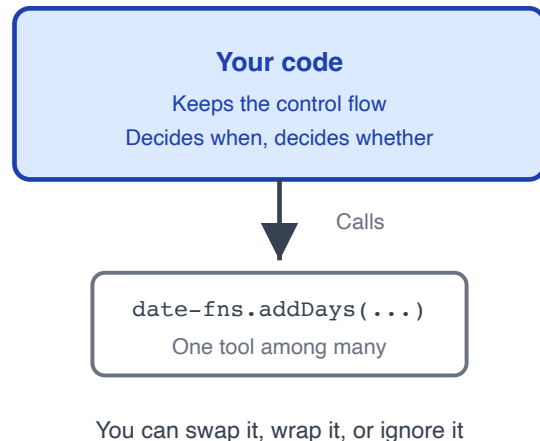
Why import anything?

- `import android.*` was about 12 million lines you did not write (Lecture 1). Most of a typical program is like that, and agents do not change it.
- Reused code has been run by many people before you. Generated code has probably only been run by you.
- Whole product families are one framework plus plugins.
 - VS Code and its extensions, browsers and theirs, Linux and its distributions.
- As an aside: should you be importing stuff? Hold on to that thought for later.

A library...

- Code you call. Your program keeps its structure and its control flow, and the library returns when it is done.
- You have used several before. For example, jqwik in Lab 2, the collections in every language, and whatever hashed passwords and minted tokens for Better Slack's login.
- A library usually leaves the rest of your program alone. Combine several, replace one, or put your own interface in front of one (the Adapter, coming up).

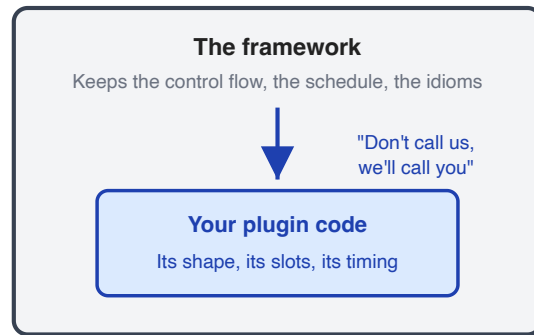
A library: you call it



... is different from a framework

- Skeleton code you customize into an application. It keeps the structure and the control flow, and calls your code at points it chose.
- You have also used several. The browser calls your page's event handlers, Express calls your route handlers, Android calls your app's lifecycle methods.
- Your code is a **plugin**, the method the framework calls is a **callback**, and the order it calls them in is the **protocol**.
- Your coding agent is one too. It calls your hooks and MCP servers, in an order it decides.

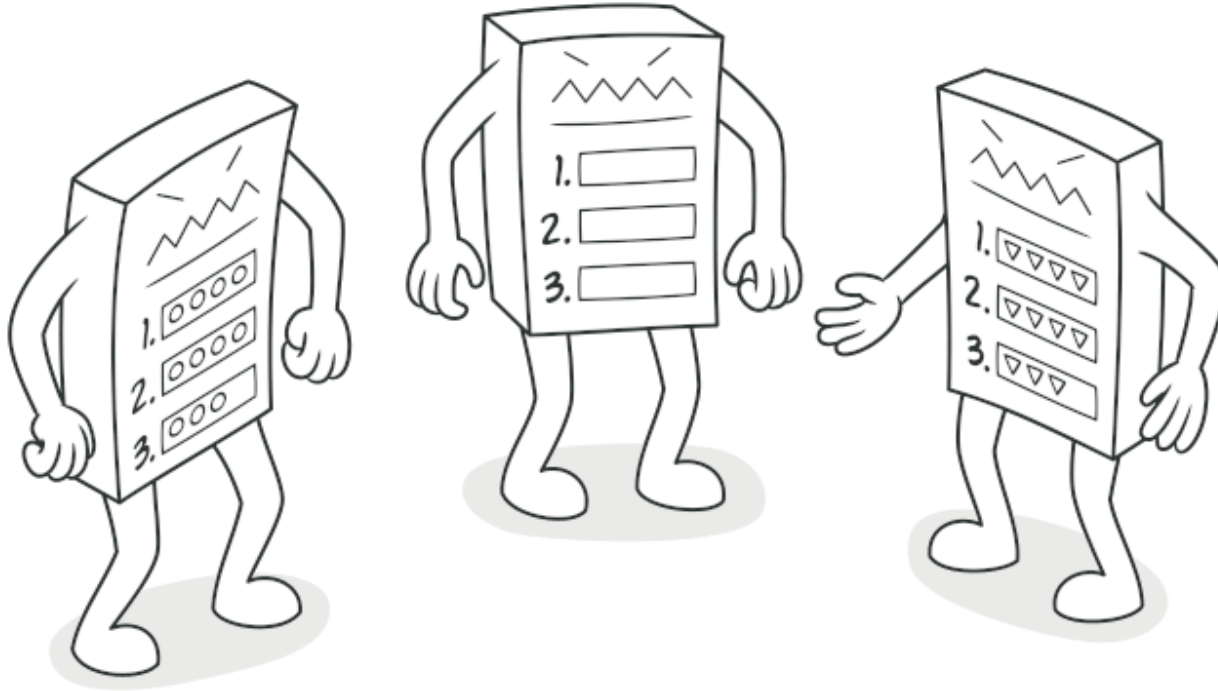
A framework: it calls you



Leaving it means rewriting your code

The framework bargain

- What you get: structure, conventions, the common decisions already made, an ecosystem.
- What you pay: your code is shaped like the framework, so leaving it means rewriting rather than relinking.
- A library is generally easy to replace. A framework usually is not, so the choice deserves a closer look/record.



refactoring.guru/design-patterns/template-method

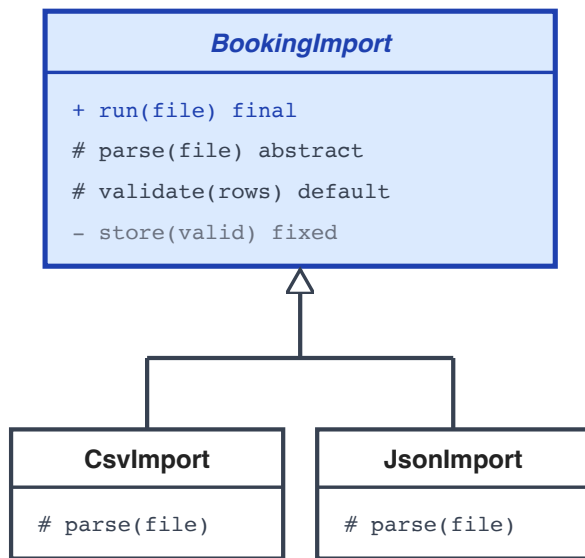
Template Method

The problem: A procedure has fixed steps in a fixed order, and some steps differ per use. The code that runs the steps should not change for each new variant.

```
abstract class BookingImport {  
    final ImportReport run(Path file) {           // the fixed steps  
        List<Row> rows = parse(file);  
        List<Booking> valid = validate(rows);  
        return store(valid);  
    }  
    protected abstract List<Row> parse(Path file); // each format fills this in  
    protected List<Booking> validate(List<Row> rows) { /* a default, but can be overridden */ }  
    private ImportReport store(List<Booking> b) { /* fixed */ }  
}  
  
class CsvImport extends BookingImport {  
    protected List<Row> parse(Path file) { /* read CSV */ }  
}
```

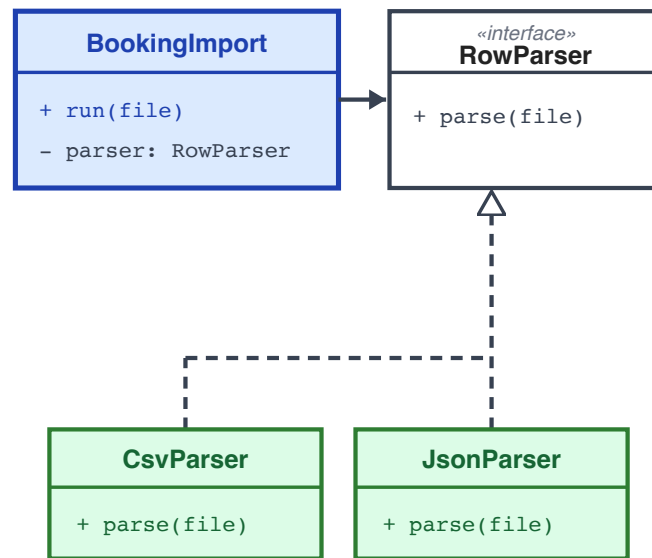
Template Method vs. Strategy

Template Method: by inheritance



The base class holds the order, the subclasses fill the steps

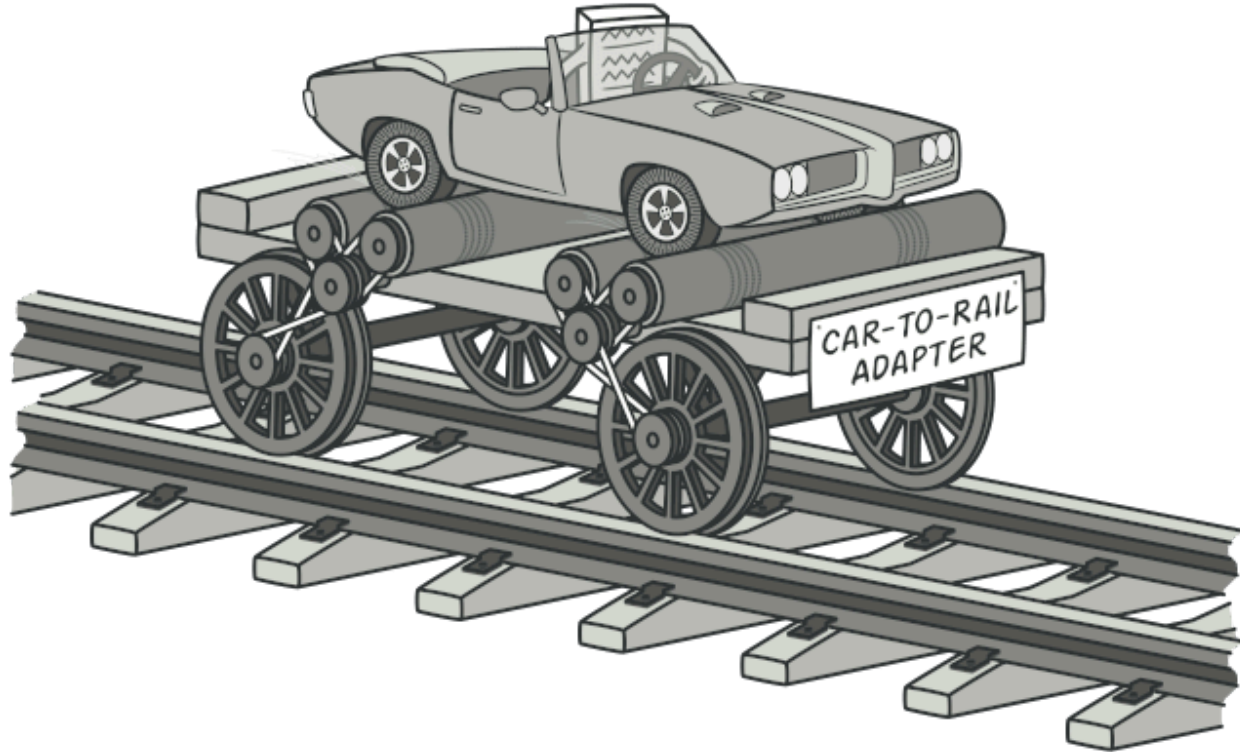
Strategy: by composition



The import holds a parser and never asks which one

Which one should you use?

- Both let one step vary. Template Method varies it by inheritance, Strategy by composition.
- Template Method has the drawbacks of inheritance.
 - The subclass is tied to the base class's order and insides (fragile base class problem), and a step cannot be swapped at run time.
- Template Method fits when the step order is fixed and not yours, as inside a framework. For example, JUnit decides setup, then test, then teardown. You write only those.
- When the varying step is a rule of your own that multiplies, needs its own tests, or is chosen at run time (a config value, a list of plugins), Strategy is usually the better fit.
- Frameworks come in both shapes.
 - Extended by **subclassing**: inheritance-based (white-box).
 - Extended by **plugging in objects that implement its interfaces**: delegation-based (black-box).

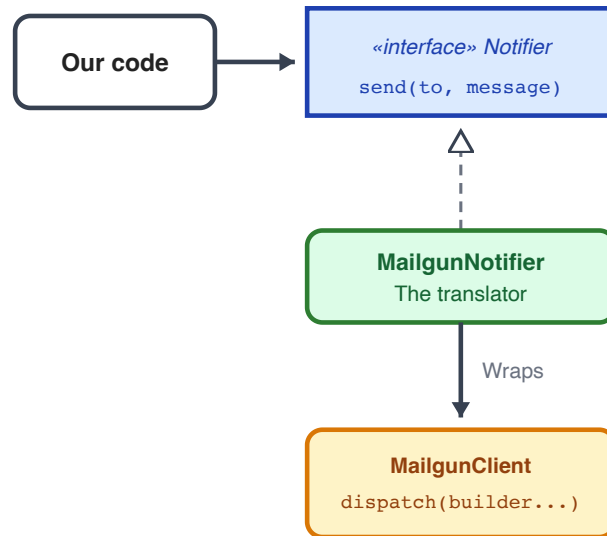


refactoring.guru/design-patterns/adapter

Adapter

The problem: Our code wants `notifier.send(to, message)`. The vendor SDK offers `MailgunClient.dispatch(payloadBuilder.build())`, and we do not own the vendor's code.

- Wrap it. A small class implements our interface and translates to theirs.
- The vendor's conventions live in one file, and swapping vendors is one new adapter.



An adapter for Mailgun

```
class MailgunNotifier implements Notifier {  
    constructor(private client: MailgunClient) {}  
  
    send(to: string, message: string) {  
        this.client.dispatch(  
            new PayloadBuilder().recipient(to).body(message).build();  
        )  
    }  
}
```

- Our code says `send(to, message)` everywhere. Mailgun's calls stay in this one file.
- Everything that uses `Notifier` can test against a fake. Only this file needs the vendor.
- But our interface offers only what we put in it.
 - For example, Mailgun can schedule a delivery. `Notifier` cannot.

Reuse it, or generate it?

- When writing code was expensive, the simple advice was "use a library, don't reinvent the wheel".
- A dependency has costs of its own. It is code you run with your permissions, written by a stranger, and it brings its own dependencies and its own updates.
- An agent can now generate a serviceable version of many small libraries in minutes, so some no longer justify a dependency.
- Either way, it is not a bad idea to briefly record the call (including the alternative, the reason, and when to revisit).

Recurring bookings need a rule

"Every Tuesday, 10 to 11, until December 15. Skip November 24."

Reuse: rrule.js

- Implements most of RFC 5545, the iCalendar recurrence standard (2009).
- On npm since 2012. About 3.7 million downloads a week.
- 687 KB, one dependency, last release November 2023.

Generate: our own rule

- A weekday, a time, an end date, a list of skipped dates.
- Perhaps forty lines and a dozen tests, to a spec we wrote.
- Covers only our case. We have to handle daylight saving time ourselves.

Which one should we use?

It depends...

- On how much of the standard you need. One weekly rule is forty lines. "Any rule a calendar app can export" is the whole RFC.
- On who maintains it. A last release three years ago is a question to ask before importing.
- On what you get right either way. Daylight saving time needs to be handled in both columns.

You don't need to weigh *every* import, by the way. Many are routine (the standard library, the framework you already chose, or a well-known library doing the job it exists for).

But, if an unknown package is pulled in for one function, or something touches security, money, or time, or if something brings a large tree of dependencies with it, think twice before saying "LGTM". (Agents can add imports liberally. Spot-check the diffs.)

Two ways agents can get this wrong

- One direction: date arithmetic reimplemented inline when the project already ships a library for it.
 - Duplication over reuse, usually from missing context.
- The other direction: a large dependency added for one function, because that import is common in the code the model learned from.
- Two reasonable review questions when a dependency changes unprompted:
 - Does a well-maintained library already do this?
 - Was this new dependency needed, or would a few lines of our own have done?

Summary

- Your API is a promise made to strangers. The surface is theirs, everything behind it is yours, and Hyrum's Law says the promise is bigger than what you wrote.
- Try to make the wrong call unwritable. Types that make illegal states unrepresentable check every caller at once, human or agent.
- Honor the contract you claim (LSP) and keep the surface small (ISP).
- Evolve by adding. Break only with a deprecation path and dates, and verify the boundary with contract tests.
- On the other side of the boundary, you are the stranger. A library is code you call, while a framework calls you.
- Adapter wraps what you do not own, and Template Method fixes the order of steps while letting a subclass fill them in.
- Reuse or generate depends on how much of the problem is domain-hard, who maintains it, and what you own either way. Record the call.

What's next

W1 A1 out	W2	W3 A1 due	W4	W5 A2 due · M1	W6	W7 A3 due	W8	W9 A4 due	W10 M2	W11	W12 A5 due	W13	W14 A6 due
--------------	----	--------------	----	-------------------	----	--------------	----	--------------	-----------	-----	---------------	-----	---------------

- **Wednesday:** Evolution, drift, and engineering memory, and how to read a codebase you did not write. A3 is due that night, 11:59 pm.
- **Wednesday night:** Assignment 4 comes out. A large codebase you have never seen.
- **Friday, Lab 7:** Direct a refactor with a characterization test first, then critique the patterns in the code.